EXPERIMENT NO: 5

Student Name: Ashwani Pandey **UID:** 23BCS11229

Branch: BE-CSE **Section/Group:** KRG-1A

Semester: 6th **Date of Performance:** 23/02/2026

Subject Name: System Design **Subject Code:** 23CSH-314

Aim

to create a scalable real-time messaging program that is akin to Facebook Messenger or WhatsApp and that facilitates group and one-to-one messaging with dependable message delivery, low latency, and high availability.

Objective

• Recognize a real-time messaging system's architecture.
• Determine the functional needs for things like media sharing, chat, and authentication.
• Determine non-functional needs, such as latency, availability, and reliability restrictions.
• Examine the trade-offs for messaging systems in the CAP theorem.
• Create WebSocket and REST APIs to facilitate chat.

Procedure

1. Studied real-world messaging systems like WhatsApp and Facebook Messenger.
2. Identified core entities such as Users, Groups, and Messages.
3. Analyzed real-time communication using WebSockets.
4. Collected functional and non-functional requirements.
5. Designed APIs for user authentication and messaging.
6. Evaluated storage and scalability requirements for large-scale systems.
7. Analyzed availability and eventual consistency trade-offs.

Functional Requirements

1. The client ought to be able to sign up and access the system.
2. One-to-one messaging should be supported by the system.
3. Group communications should be supported by the system.
4. Both text and media message sending should be supported by the system.
5. The entire communication history should be preserved by the system.
6. The client should be able to view and read receipts for deliveries.
7. Real-time message reception is required.

Core Entities of the System

- Users
- Groups

- Messages

API Design

1. **User Registration API**: POST /user/register
2. **User Login API**: POST /user/login

One-to-One Messaging

A. Send Message (WebSocket): WS /messages/send
B. Get Chat List: GET /chat/{user_id}(Pagination Supported)
C. Get Messages of Specific Chat: GET /messages/{user_id}/{receiver_id}

Group Messaging

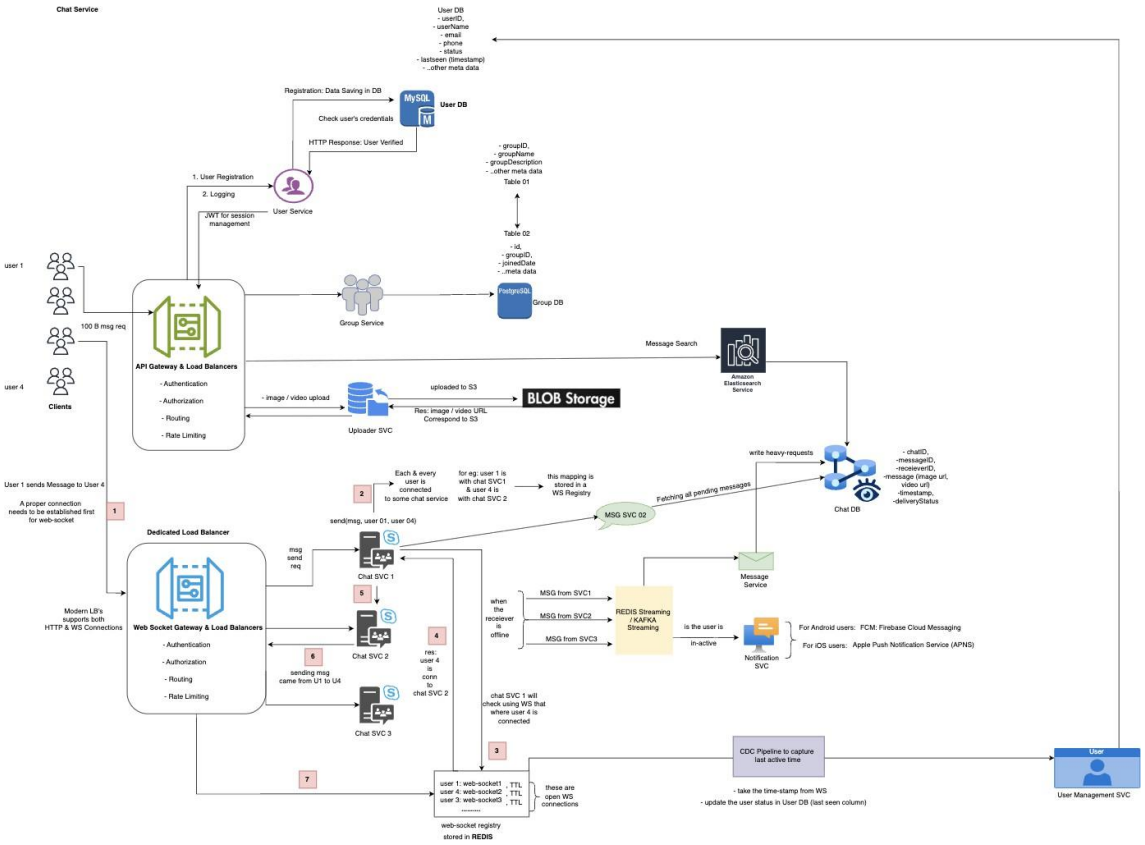
A. Create Group: POST /groups/create
B. Send Group Message (WebSocket): WS /messages/send
C. Get Group Chats: GET /groups/{group_id}(Pagination Supported)
D. Add Member: POST /groups/{group_id}/add
E. Remove Member: DELETE /groups/{group_id}/remove

Non-Functional Requirements

1. System should handle extremely large scale traffic (millions to billions of messages per day) requiring approximately 100 TB storage.
2. High Availability should be prioritized under CAP theorem.
3. System should follow Eventual Consistency model.
4. End-to-end latency should be approximately 200–300 ms.
5. System should be highly reliable with no message loss or packet drop.
6. Horizontally scalable distributed architecture.

High Level Design (HLD)

The system consists of Client Applications (Mobile/Web), API Gateway, Authentication Service, Messaging Service (WebSocket Servers), Group Service, Media Service, Message Queue, Distributed Databases, Cache Layer (Redis), and Object Storage for media files. Messages are delivered through WebSocket servers for real-time communication and stored in distributed databases for persistence.



Learning Outcomes

- Designed a scalable real-time messaging system.
- Identified functional and non-functional requirements.
- Designed REST and WebSocket APIs.
- Understood high availability and eventual consistency trade-offs in messaging systems.